THE JOE TEAM REFERENCE BOOK

JoeLang

Object-Facing Programming for Humans

Version 0.9 — Working Manual

What this book is for. This manual explains how to read, write, project, and debug JoeLangprograms in the planned online IDE. It is written for Joe Team members who want a practical shared notation before building the console.

Framing. At the beginning, one point should be plain: this is a creative parody reference built from the Joelib archive and a new JoeLang specification. It does not alter ownership or meaning of the cited source materials.

The promise. Start with the thing in your program. Give it state. Give it behavior. Send it a message. Let the console show you what happened.

Prepared for the Joe Team

November 7, 2026

Contents

1 Welcome to JoeLang 3

2 Your First Program 4

3 The Object-Facing Mental Model 5

4 Keyword Field Guide 6

5 Declaring Joebjects 7

6 State: What a Joebject Has 8

7 Behaviors and Messages 9

8 Making Decisions with `when` 10

9 Collections and Repetition 11

10 Text, Numbers, and Values 12

11 Creating and Connecting Joebjects 13

12 Joelements and Approved Imports 14

13 Reading the Projection Pane 15

14 Console, Inspector, and Run Records 16

15 Errors You Will Meet 17

16 Style Guide for Joe Team Programs 18

17 A Small Complete Example 19

18 Exercises for the Joe Team 20

19 Quick Reference 21

20 Source Notes and Credits 22

21 Closing Note **23**

Sources **24**

1 Welcome to JoeLang

JoeLangis a planned language for making small programs, interactive sketches, tools, and demos in an online console. It projects to JavaScript, but you normally begin with a more direct question: *what thing is this program about?* That thing is a Joebject.

The language was designed from patterns already visible in Joelib. Joeglish work emphasizes dependable two-way word transformation. The Joelements project keeps familiar chemical symbols while making Joe names visible. The archive organizes information as a growing collection of research records, literature, visual material, and regional lists.[2, 5, 1] JoeLangtakes the same practical attitude: keep execution understandable, make the Joe-facing language readable, and leave evidence visible.

You do not need to know JavaScript before learning this manual. You will eventually be able to inspect the JavaScript projection, but the first skill is to describe an object and what it can do.

Team tip

Read every program as a small cast list. A Joebject is a named actor. `has` describes what it carries. `does` describes what it can do. A message asks it to do one of those things.

This book uses the words *program*, *source*, and *Joe file* interchangeably. A complete program may contain one or more Joebjects plus a few messages that begin the action.

2 Your First Program

Here is a complete small program. Type it in the Source pane of the online IDE, then choose *Project and Run*.

```
joebject Greeter {
  has name = "Joe Team"

  does welcome() {
    show "Welcome, " + name + "!"
  }
}

newjoe Greeter as guide
guide.welcome()
```

The program produces:

```
Welcome, Joe Team!
```

Read it from top to bottom.

1. `joebject Greeter` creates the kind of thing this program knows about.
2. `has name` gives every Greeter a remembered piece of data.
3. `does welcome()` gives every Greeter an action.
4. `newjoe Greeter as guide` creates one Greeter named `guide`.
5. `guide.welcome()` sends the `welcome` message to that object.

Central rule

The dot means "send this message to that receiver." In `guide.welcome()`, `guide` is the receiver and `welcome` is the behavior being requested.

When you run a program, the console shows four useful views: your Joe source, the projected JavaScript, the output, and a Joebject Inspector. You are encouraged to inspect the projection, but you do not need to edit it to use the language.

3 The Object-Facing Mental Model

Object-facing programming is not a test you need to pass. It is a way of keeping programs easy to explain.

Consider a timer. In a procedure-first design, a file might begin with variables and scattered functions. In JoeLang, begin with the timer itself:

```
joebject StudyTimer {
  has minutes = 25
  has running = falsejoe

  does begin() {
    running = truejoe
    show "Timer started."
  }

  does stop() {
    running = falsejoe
    show "Timer stopped."
  }
}
```

The reader immediately sees the object, its remembered state, and its behavior. This is the main benefit of the design. It does not forbid small helper calculations; it asks you to place those calculations inside the object that owns their meaning.

Question	JoeLang answer
What is this program about?	Name a Joebject.
What does it remember?	Declare it with `has`.
What can it do?	Declare it with `does`.
Who starts the action?	Create it with `newjoe` and send a message.
Who may change this value?	Its owning Joebject, through one of its behaviors.

4 Keyword Field Guide

The following words have fixed meanings in version 0.9. Do not use them as names for your own Joebjects, state, or behaviors.

Keyword	Use	Meaning
`joebject`	declaration	Begins a named object definition.
`has`	state	Gives a Joebject an owned value.
`does`	behavior	Defines an action the Joebject can receive.
`newjoe`	creation	Creates an instance of a Joebject type.
`as`	naming	Gives a created instance a local name.
`when`	decision/event	Runs a block when a condition is true.
`otherwise`	decision	Supplies the other path for a `when`.
`give`	return	Sends a value back from a behavior.
`show`	output	Writes a value to the console.
`ask`	import	Requests an approved Joelement or package binding.
`from`	import	States where an approved binding comes from.
`each` / `in`	loop	Repeats an action for values in a collection.
`truejoe` / `falsejoe`	logic	The two logical values.
`nonejoe`	empty value	A deliberate “no value here” marker.

The terms borrow the archive’s established Joe-facing naming practice, but these exact language meanings are part of the new JoeLang specification. Joeglish source work is especially relevant because it treats a Joe spelling as a disciplined mapping rather than random decoration.[3, 4]

5 Declaring Joebjects

Use `joebject` to make a blueprint. A blueprint is not yet a running object. It is a description from which you can create objects later.

```
joebject Notebook {
  has title = "Untitled"
  has pages = 0

  does add_page() {
    pages = pages + 1
    give pages
  }
}
```

Names should begin with a letter. By convention, use a capitalized singular name for a Joebject type, a lower-case name for an instance, and lower-case words joined by underscores for behaviors and state.

Prefer	Avoid	Reason
`Notebook`	`notebook_thing_2`	A type should state what it is.
`study_timer`	`x`	An instance name should say its role.
`add_page`	`DoTheThing`	Behaviors are easier to scan as actions.
`page_count`	`p`	State should identify what is remembered.

Watch for this

A Joebject declaration does not run its behaviors. You must create an instance with `newjoe`, then send it a message.

6 State: What a Joebject Has

State is data that belongs to one Joebject. Declare it near the top of the Joebject with `has`. A value can be text, a number, a logical value, a list, a record, or `nonejoe`.

```
joebject Lantern {
  has color = "blue"
  has charge = 3
  has lit = falsejoe
}
```

Inside a behavior of `Lantern`, you can read and change `color`, `charge`, and `lit` directly. Outside the Joebject, ask the lantern to do something instead of reaching into its private state.

```
joebject Lantern {
  has charge = 3

  does use_one_charge() {
    when charge > 0 {
      charge = charge - 1
    }
    give charge
  }
}
```

Central rule

State belongs to its owner. A behavior may update its own state. Another Joebject requests a behavior; it does not perform a hidden state update from a distance.

This rule makes bugs easier to locate. When `charge` changes, there is a small, predictable group of places to inspect: behaviors owned by `Lantern`.

7 Behaviors and Messages

Use `does` to define an action. Put inputs inside parentheses. Use `give` when the behavior should send a value back.

```
joebject Scoreboard {
  has score = 0

  does add(points) {
    score = score + points
    give score
  }
}

newjoe Scoreboard as board
show board.add(10)
```

The message `board.add(10)` asks `board` to run its `add` behavior with `10` as the value of `points`. The behavior changes owned state, then gives the new score back to the caller.

Messages can have no inputs, one input, or several ordered inputs:

```
lamp.turn_on()
board.add(10)
map.move_to(4, 7)
```

Team tip

If you struggle to name a behavior, write a short sentence: “The map can move to a location.” The behavior name is often `move_to`; the receiver is `map`.

8 Making Decisions with `when`

Use `when` to choose an action based on a true or false condition. Use `otherwise` when you need an alternate path.

```
joebject Door {
  has locked = truejoe

  does enter() {
    when locked {
      show "The door is locked."
    } otherwise {
      show "Welcome in."
    }
  }

  does unlock() {
    locked = falsejoe
  }
}
```

Conditions use familiar comparisons:

```
when score >= 100 { show "Level complete" }
when name == "Joe" { show "Recognized" }
when not ready { show "Still preparing" }
```

Use a condition that is easy to say aloud. “When the door is locked, show the locked message” is clearer than a complicated expression that combines several unrelated tests. If you do need multiple tests, group them in a behavior with a meaningful name such as `can_enter()`.

9 Collections and Repetition

A collection is a group of values. In the base language, write a list with square brackets and a record with braces and named fields.

```
has regions = ["Kanjoe", "Joeto", "Joenn"]
has profile = { name: "Ari", role: "researcher" }
```

Use `each` and `in` to do the same behavior for every value in a collection.

```
joebject RollCall {
  has names = ["Ari", "Bo", "Cy"]

  does greet_everyone() {
    each person in names {
      show "Hello, " + person
    }
  }
}
```

The collection is read from left to right. The name after `each` is a temporary value available within the braces. It is not state and disappears when that repetition ends.

Regional Joekemon lists in Joelib provide a useful model for grouped names: a region can organize a collection without changing what each named item is.[6] In the IDE, the same idea will organize optional packages and example collections.

10 Text, Numbers, and Values

Text is written between quotes. Numbers are written directly. Logical values are `truejoe` and `falsejoe`. Use `nonejoe` to show that a value is intentionally not available yet.

Kind	Example	When to use it
Text	`"Joe team"`	Names, labels, messages, written content.
Number	`42`	Counts, amounts, positions, scores.
Logic	`truejoe`	A yes/no state or condition.
Empty	`nonejoe`	A value that has not been chosen or found.
List	`["red", "blue"]`	An ordered group.
Record	`{ name: "Ari" }`	Several named values that belong together.

Use `+` to join text or add numbers. The IDE will warn you when it cannot tell which one you meant. Keep values of the same kind together where possible.

```
show "Pages: " + pages
has total = price + tax
```

The language's display words are deliberately separate from values. A string such as `"Joeglish"` is data; the compiler does not rewrite the letters inside it during normal projection. This protects stored names, quotations, and messages from accidental transformation. The practice follows the translator's careful separation of word tokens from non-word content.[4]

11 Creating and Connecting Joebjects

Use `newjoe` to create an instance. Use `as` to name it so the rest of the program can send it messages.

```
newjoe StudyTimer as timer
timer.begin()
```

Joebjects can work together by sending messages. Here, a `Session` asks its own timer to begin.

```
joebject Session {
  has timer = nonejoe

  does prepare() {
    timer = newjoe StudyTimer
  }

  does start() {
    when timer == nonejoe {
      prepare()
    }
    timer.begin()
  }
}
```

The practical rule is simple: keep the relationship visible. If `Session` owns the timer it uses, store it in `timer`. If it only needs to use a timer temporarily, receive that timer as a behavior input instead.

Watch for this

Do not create a long hidden chain of objects that all change each other. Prefer one message at a time: `session.start()` asks the session to decide what must happen next.

12 Joelements and Approved Imports

A Joelement is an approved library capability with a Joe-facing name, a clear description, and an inspectable host implementation. Use `ask` and `from` to request one.

```
ask clock from "Joexyjin"

joebject Bell {
  does ring_later() {
    clock.after(1000, "ring")
  }
}
```

The exact initial library catalog will be set by the IDE build. The source rule does not change: an import must name both the local binding and the approved Joelement package. The console will show the package's permissions, host capability, and documentation before first use.

This naming model is informed by the Joelements interface: it presents a Joe identity while retaining an original anchor for clarity.[5] In a program, the source-level Joelement name is the friendly front door; the generated JavaScript and module record remain available for inspection.

Central rule

Imports are requests, not hidden powers. If a program has not asked for a capability, it does not receive it.

13 Reading the Projection Pane

The IDE's Projection pane displays JavaScript created from your source. It is there to build trust and help with learning, not to make every user a JavaScript programmer.

For example, this JoeLang behavior:

```
does add(points) {
  score = score + points
  give score
}
```

will project to a JavaScript method with a return statement. The exact generated spelling can vary between compiler versions, but the behavior must remain the same: accept `points`, update the owning score, then return it.

Use the Projection pane in three situations:

1. You want to confirm that a source word became the host feature you expected.
2. You receive a runtime error and need to see which source line generated the failing host code.
3. You are learning how a JavaScript idea relates to a JoeLang idea.

The projection layer is inspired by Joeglish translation work, which documents direction, mapping, casing, and a round-trip result instead of treating transformation as mysterious.[2, 4] The IDE will similarly identify the source span, structural rule, and generated span for a selected token.

14 Console, Inspector, and Run Records

Every run makes a small record. The record contains:

- the source text and language version;
- the projection version and generated-code identity;
- console output and returned values;
- a snapshot of visible Joebject state; and
- any warnings or errors.

The Joebject Inspector is especially useful for beginners. It shows a tree such as:

```
guide : Greeter
  name: "Joe Team"
  behaviors: welcome
```

If your output is surprising, look first at the inspector. Ask: did the correct object receive the message? Did its remembered state have the value I expected? Did the behavior return a value or only show one?

The UI should make keyboard navigation and text copying available for every pane. JOESII demonstrates an archive-facing visual language built around displayed characters and signal-like motion; the console can take visual inspiration from it without replacing readable text with effects.[7]

15 Errors You Will Meet

Errors are evidence. Read the first line, then use the source span and suggested next action.

Diagnostic	Usually means	First fix to try
Unknown Joebject	The blueprint name was misspelled or not declared.	Check the spelling after `joebject`.
Unknown receiver	You sent a message to a name that has no current object.	Create it with `newjoe` or pass it in.
Behavior not observed on receiver	The object exists, but it has no behavior with that name.	Check the `does` declaration and message spelling.
State may only change through its owner	Code outside the object tried to change private state.	Add a behavior to the owner and send it a message.
Expected closing brace	A block was opened but not closed.	Match each `{` with a `}`.
Projection mapping is not reversible	A display transformation would collide with another name.	Keep the stable identifier or choose another name.
Capability not approved	A library action was requested without an approved import.	Add an `ask ... from ...` statement or choose a local operation.

Team tip

Fix the earliest error first. A missing brace near the top can cause many later messages that disappear as soon as the first mistake is corrected.

16 Style Guide for Joe Team Programs

Good JoeLang source should be easy to narrate to another team member.

Name things by their role

Prefer `research_log`, `signal_reader`, and `study_timer` over one-letter names. The additional characters save time when someone else reads the program later.

Keep behaviors short

A behavior should normally do one recognizable job. If it begins to handle setup, validation, calculation, display, and cleanup all at once, divide it into smaller named behaviors.

Show, then give

Use `show` for a message a human should see. Use `give` for a value another part of the program needs. A behavior may use both, but do not assume console output is a return value.

Keep state close to its owner

If a value belongs to the lantern, the timer, or the research log, declare it inside that Joebject. Do not build a hidden pile of unrelated global values.

Explain the unusual part

Use a short comment before a surprising choice. Comments are never projected through the Joeglish dictionary, so they remain safe places for straightforward explanation.

17 A Small Complete Example

This example combines a Joebject, state, a behavior input, a decision, a list, repetition, output, and a returned value.

```
joebject FieldLog {
  has entries = []

  does record(note) {
    entries = entries + [note]
    show "Recorded: " + note
    give entries
  }

  does review() {
    when entries == [] {
      show "No observations yet."
    } otherwise {
      each entry in entries {
        show "- " + entry
      }
    }
  }
}

newjoe FieldLog as log
log.record("Signal received")
log.record("Projection verified")
log.review()
```

Walk through the program before running it. `log` is an instance of `FieldLog`. It begins with an empty list. Each `record` message makes a new list containing the old entries plus the new note. The `review` message decides whether there is anything to show, then prints each entry.

The best habit is to predict the output before pressing Run. If the actual output differs, use the inspector to check `entries` after each message.

18 Exercises for the Joe Team

Try these in the IDE once the first console version is available. Write the source yourself before looking at the suggested shape.

1. Create a `Badge` Joebject with a `name` state and a `display()` behavior.
2. Create a `Counter` with a number state and an `increase(by)` behavior that gives the new number back.
3. Add a `reset()` behavior to the Counter.
4. Create a `Checklist` that keeps a list of tasks and shows them one per line.
5. Create a `Door` with locked state. Add `lock()`, `unlock()`, and `enter()` behaviors.
6. Ask for one approved time-related Joelement and make a Joebject schedule a message.
7. Deliberately send a message to a nonexistent behavior. Read the diagnostic, then repair the spelling.
8. Select one identifier in the Source pane and follow its path in the Projection pane.

When sharing an exercise result, send three things: the source, the console output, and the run record. That is enough for another team member to reproduce most problems without guessing.

19 Quick Reference

```
// A Joebject blueprint
joebject Name {
  has state_name = value

  does behavior_name(input) {
    when condition {
      show "visible message"
      give value
    } otherwise {
      show "other path"
    }
  }
}

// Create and use it
newjoe Name as instance
instance.behavior_name(value)

// Repeat over a list
each item in items {
  show item
}

// Request an approved library feature
ask local_name from "JoelementName"
```

Central rule

The five questions to ask when reading a program are: What is the Joebject? What state does it have? What behavior is being requested? Which object receives the message? What value or output should result?

20 Source Notes and Credits

The new language rules in this manual are a Joe Team specification. Their design cues come from the Joelib materials listed below. The credits use the attributions supplied for this manual:

Contributor	**Referenced work**
Derek Li	Joelements, *The Great Joetsby*, and the Joe Wiki.
Joe Jin (the central figure himself)	Joeglish Grammar.
Wayne Xu	Joeglish Translator, JOESII, and JOESII art.
Matthew DeRoma	Joekemons.
Emilia Marcucci	*Joelet Speech*.

The repository README is the index to those documents and projects. It describes Joelib as a projection of the Joe universe and links the primary materials used for this manual.[1] The material supports the language's vocabulary, archival attitude, reversible translation inspiration, and interface motifs; it does not itself define the new JoeLang grammar.

21 Closing Note

Use JoeLangto make programs that another person can read as a conversation among named things. Keep the source clear, inspect the projection when it helps, and leave a useful run record for the next person.

This final note repeats the scope plainly: JoeLang and this manual are parody design work for the Joe Team. The source archive remains credited to the contributors named above and to the Joelib repository.

References

[1] N0TD3R3K/Joelib contributors. *Joelib README*. Repository index to the Joe Wiki, Joeglish Grammar, *The Great Joetsby*, Joelements, Joekemons, Joelet Speech, JOESII, and Joegle Translate. https://github.com/N0TD3R3K/Joelib/blob/main/README.md. Accessed 2026-11-06.

[2] Wayne Xu (attribution supplied by the Joe Team). *Joegle Translate README*. English–Joeglish interface and deterministic transformation description. https://github.com/N0TD3R3K/Joelib/blob/main/joeglish_translator/README.md. Accessed 2026-11-06.

[3] Wayne Xu (attribution supplied by the Joe Team). *Joeglish dictionary generator*. Seed mappings, candidate rules, collision checks, and bijectivity assertions. https://github.com/N0TD3R3K/Joelib/blob/main/joeglish_translator/scripts/generateDictionary.js. Accessed 2026-11-06.

[4] Wayne Xu (attribution supplied by the Joe Team). *Joeglish translation engine*. Tokenization, casing preservation, directional lookup, and round-trip computation. https://github.com/N0TD3R3K/Joelib/blob/main/joeglish_translator/src/engine/joeglishEngine.ts. Accessed 2026-11-06.

[5] Derek Li (attribution supplied by the Joe Team). *The Joeriodic Table of Joelements*. Interactive interface and structured element dataset. https://github.com/N0TD3R3K/Joelib/tree/main/joelements. Accessed 2026-11-06.

[6] Matthew DeRoma (attribution supplied by the Joe Team). *Joekemon regional TSV collections*. https://github.com/N0TD3R3K/Joelib. Accessed 2026-11-06.

[7] Wayne Xu (attribution supplied by the Joe Team). *JOESII and JOESII art*. Canvas-based ASCII presentation. https://github.com/N0TD3R3K/Joelib/tree/main/joesii. Accessed 2026-11-06.

[8] Emilia Marcucci (attribution supplied by the Joe Team). *Joelet Speech*. Repository PDF and README-linked document. https://github.com/N0TD3R3K/Joelib/blob/main/Joelet%20Speech.pdf. Accessed 2026-11-06.

[9] Joe Jin (the central figure himself; attribution supplied by the Joe Team). *Joeglish Grammar*. Linked from the Joelib README. https://github.com/N0TD3R3K/Joelib/blob/main/README.md. Accessed 2026-11-06.

[10] Derek Li (attribution supplied by the Joe Team). *Joelib Wiki: Home*. https://github.com/N0TD3R3K/Joelib/wiki. Accessed 2026-11-06.